

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics
```

```
In [10]: df = pd.read_csv(r"D:\Pradnya\TE\DSBDA\Practicals\new\boston-housing\train.csv")
print("First 5 rows:\n", df.head())
```

First 5 rows:

	ID	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	\
0	1	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	
1	2	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	
2	4	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	
3	5	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	
4	7	0.08829	12.5	7.87	0	0.524	6.012	66.6	5.5605	5	311	

	ptratio	black	lstat	medv
0	15.3	396.90	4.98	24.0
1	17.8	396.90	9.14	21.6
2	18.7	394.63	2.94	33.4
3	18.7	396.90	5.33	36.2
4	15.2	395.60	12.43	22.9

```
In [11]: print("\nDataset Info:\n")
print(df.info())

print("\nStatistical Summary:\n")
print(df.describe())
```

Dataset Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 333 entries, 0 to 332

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	ID	333 non-null	int64
1	crim	333 non-null	float64
2	zn	333 non-null	float64
3	indus	333 non-null	float64
4	chas	333 non-null	int64
5	nox	333 non-null	float64
6	rm	333 non-null	float64
7	age	333 non-null	float64
8	dis	333 non-null	float64
9	rad	333 non-null	int64
10	tax	333 non-null	int64
11	ptratio	333 non-null	float64
12	black	333 non-null	float64
13	lstat	333 non-null	float64
14	medv	333 non-null	float64

dtypes: float64(11), int64(4)

memory usage: 39.2 KB

None

Statistical Summary:

	ID	crim	zn	indus	chas	nox \
count	333.000000	333.000000	333.000000	333.000000	333.000000	333.000000
mean	250.951952	3.360341	10.689189	11.293483	0.060060	0.557144
std	147.859438	7.352272	22.674762	6.998123	0.237956	0.114955
min	1.000000	0.006320	0.000000	0.740000	0.000000	0.385000
25%	123.000000	0.078960	0.000000	5.130000	0.000000	0.453000
50%	244.000000	0.261690	0.000000	9.900000	0.000000	0.538000
75%	377.000000	3.678220	12.500000	18.100000	0.000000	0.631000
max	506.000000	73.534100	100.000000	27.740000	1.000000	0.871000

	rm	age	dis	rad	tax	ptratio \
count	333.000000	333.000000	333.000000	333.000000	333.000000	333.000000
mean	6.265619	68.226426	3.709934	9.633634	409.279279	18.448048
std	0.703952	28.133344	1.981123	8.742174	170.841988	2.151821
min	3.561000	6.000000	1.129600	1.000000	188.000000	12.600000
25%	5.884000	45.400000	2.122400	4.000000	279.000000	17.400000
50%	6.202000	76.700000	3.092300	5.000000	330.000000	19.000000
75%	6.595000	93.800000	5.116700	24.000000	666.000000	20.200000
max	8.725000	100.000000	10.710300	24.000000	711.000000	21.200000

	black	lstat	medv
count	333.000000	333.000000	333.000000
mean	359.466096	12.515435	22.768769
std	86.584567	7.067781	9.173468
min	3.500000	1.730000	5.000000
25%	376.730000	7.180000	17.400000
50%	392.050000	10.970000	21.600000

```

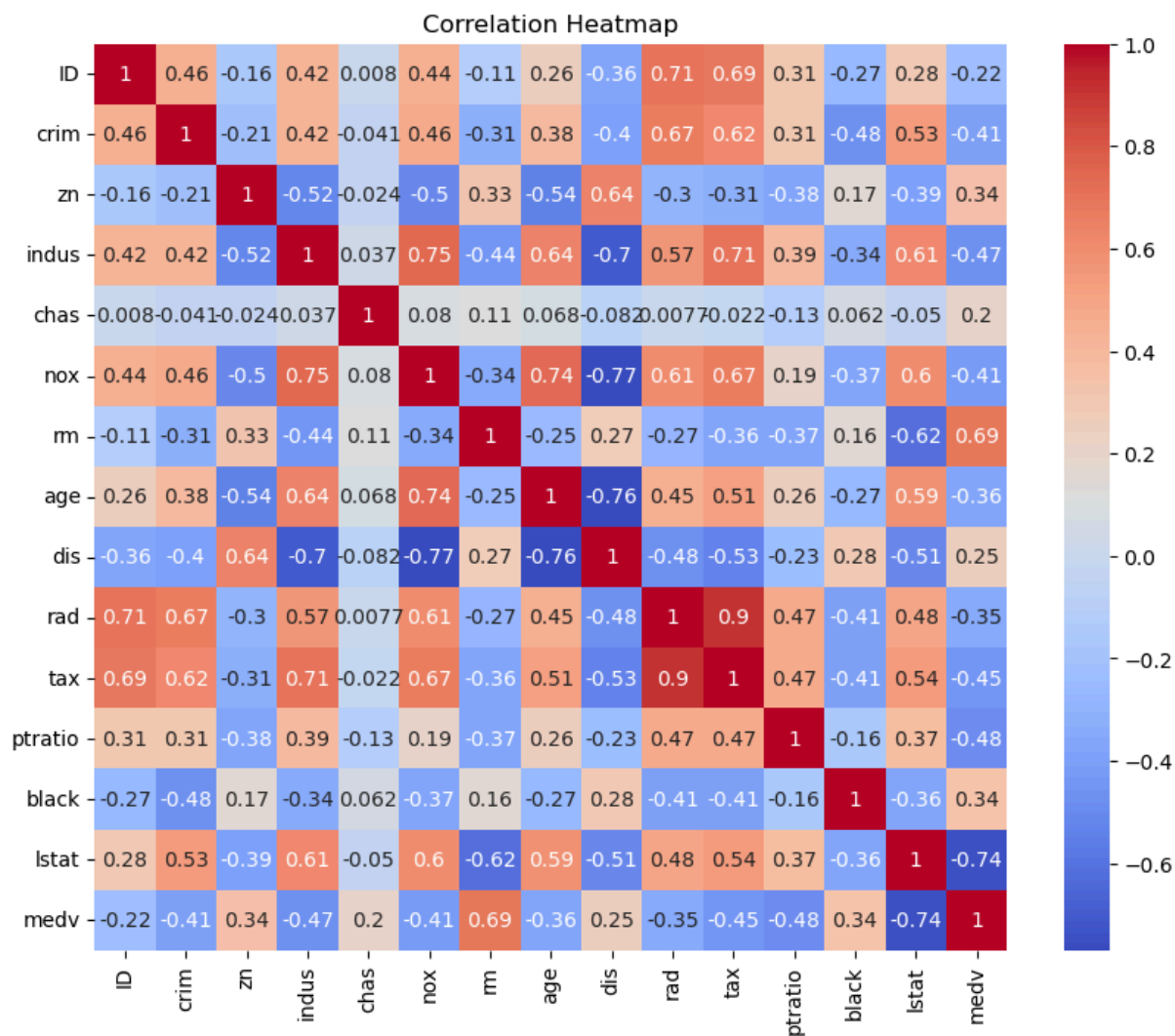
75%    396.240000    16.420000    25.000000
max     396.900000    37.970000    50.000000

```

```

In [12]: plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

```



```

In [15]: target_col = 'MEDV' if 'MEDV' in df.columns else 'medv'
X = df.drop(target_col, axis=1)
y = df[target_col]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
coeff_df = pd.DataFrame(model.coef_, X.columns, columns=['Coefficient'])
print("\nModel Coefficients:\n", coeff_df)

```

Model Coefficients:

	Coefficient
ID	-0.004902
crim	-0.070217
zn	0.061337
indus	-0.024444
chas	4.174129
nox	-14.787057
rm	3.397953
age	-0.014945
dis	-1.863079
rad	0.421680
tax	-0.013171
ptratio	-0.734327
black	0.008077
lstat	-0.637599

```
In [16]: y_pred = model.predict(X_test)
print("\nModel Evaluation:")
print("Mean Absolute Error:", metrics.mean_absolute_error(y_test, y_pred))
print("Mean Squared Error:", metrics.mean_squared_error(y_test, y_pred))
print("Root Mean Squared Error:", np.sqrt(metrics.mean_squared_error(y_test, y_pred)))
print("R2 Score:", metrics.r2_score(y_test, y_pred))
```

Model Evaluation:

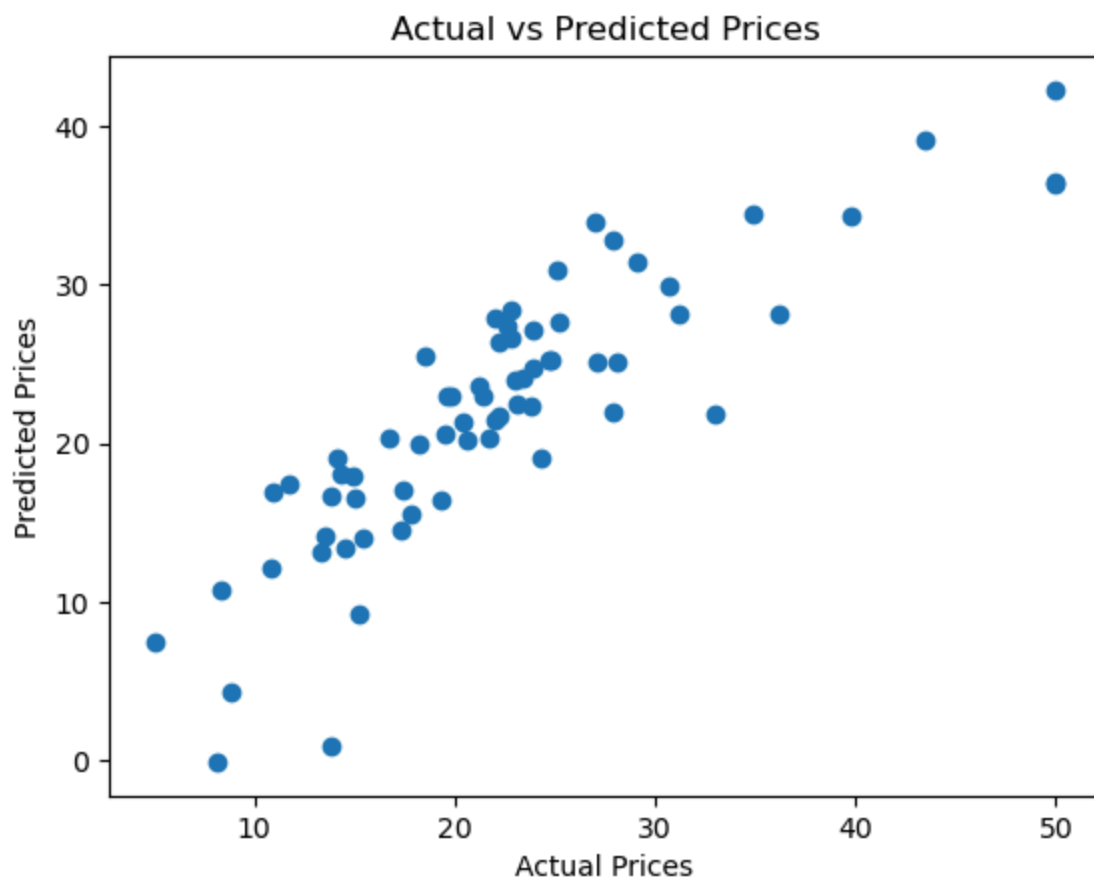
Mean Absolute Error: 3.6594858160279498

Mean Squared Error: 23.411701826598417

Root Mean Squared Error: 4.8385640252660105

R2 Score: 0.7398653051224349

```
In [17]: plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted Prices")
plt.show()
```



In []: